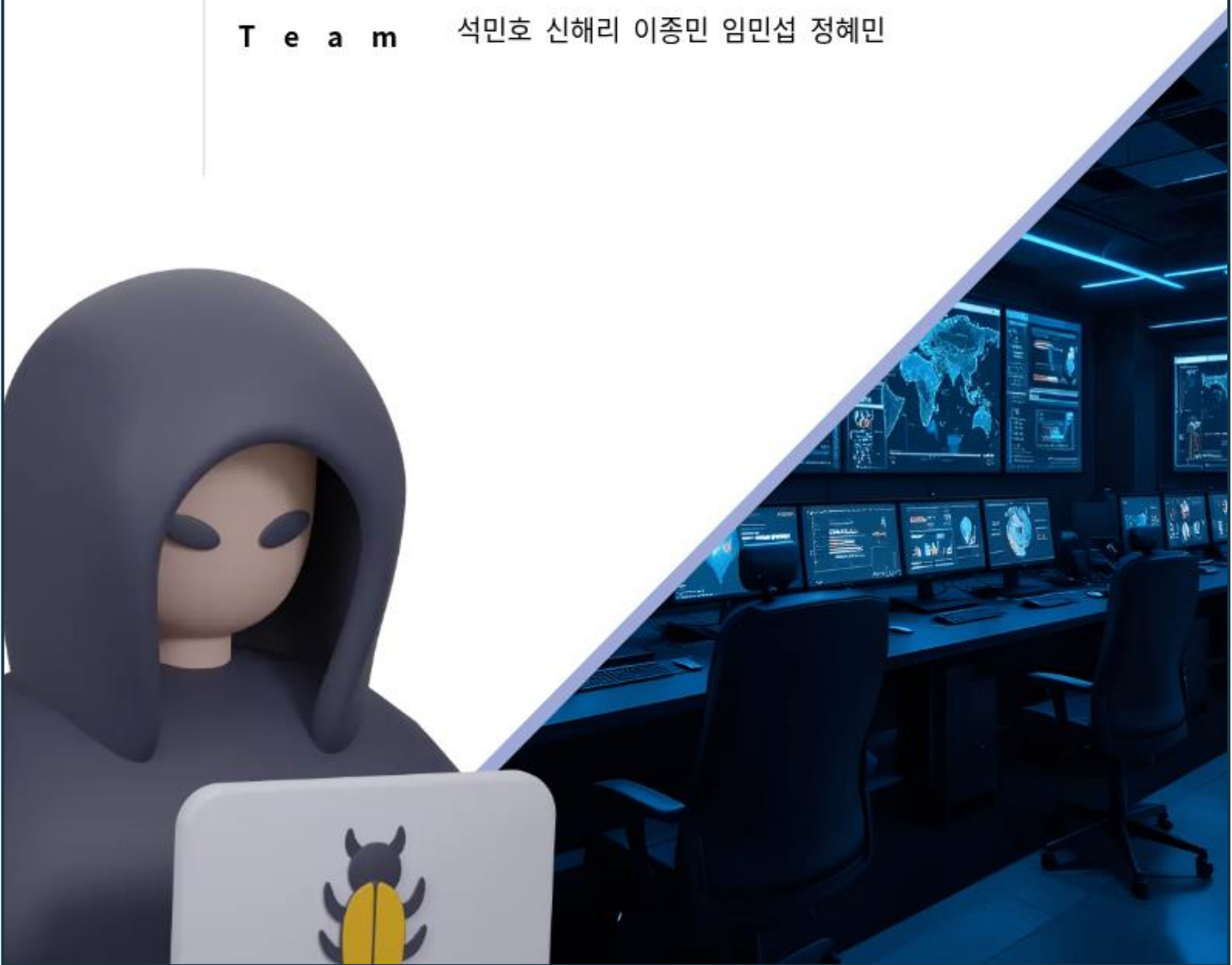
TERRAFORM을 활용한 AWS 기반 WORDPRESS 구축
-고가용성 AUTOSCALING과 WAF 보안 강화-
계획서

Period 2025.12.17 ~ 2026.01.02

Team 석민호 신해리 이종민 임민섭 정혜민



목차

1. 서론 4
1.1. 프로젝트 개요 및 목적 4
1.2. 프로젝트 추진 계획 5
1.3. 단계별 산출물 5
2. 구축 일정 및 단계 6
2.1. 프로젝트 수행 조직도 6
3. 기술 및 도구 7
3.1. Terraform 7
3.2. AWS 서비스 7
3.2.1. 인프라 기초 구성 7
3.2.2. 고가용성 확보 7
3.2.3. 보안 강화 8
3.3. WEB Service 8
4. 요구사항 분석 9
4.1. 요구사항 제안 요청서 (RFP)와 기술적 대응 방안 9
4.1.1. 인프라 관련 요구사항 9
4.1.2. 고가용성 관련 요구사항 10
4.1.3. 보안 및 운영 관련 요구사항 10
5. 구성 11
5.1. 전체 아키텍처 설계도 11
5.2. AWS 인프라 구성 설계서 12
5.2.1. 인프라 구성 12
5.2.2. 고가용성 확보 17
5.2.3. 보안 강화 20
6. 결론 및 목표 23

Copyright ⓒ 2025 Team_JS. All rights reserved.

표

[표 1] 프로젝트 개요 및 목적 4
[표 2] 단계별 산출물 5
[표 3] 인프라 관리 및 자동화 도구 7
[표 4] AWS 인프라 기본 구성 요소 7
[표 5] 고가용성 및 트래픽 처리 구성 요소 7
[표 6] 보안 처리 구성 요소 8
[표 7] web service 구성 요소 8
[표 8] 인프라 기초 요구사항 9
[표 9] 고가용성 관련 요구사항 10
[표 10] 보안 및 운영 관련 요구사항 10
[표 11] 아키텍쳐 설계도 설명 11
[표 12] IaC 기반 인프라 구축 구성 12
[표 13] 리전 구축 구성 12
[표 14] VPC 구축 구성 12
[표 15] 서브넷 구축 구성 13
[표 16] Internet gateway 구축 구성 13
[표 17] Route Table 구축 구성 14
[표 18] 도메인 구축 구성 14
[표 19] RDS 구축 구성 15
[표 20] RDS 연동 구성 16
[표 21] EC2 인스턴스 구축 구성 16
[표 22] AMI 구축 구성 17
[표 23] 가용 영역 분산 구성 17
[표 24] TG 구축 구성 18
[표 25] Template 구축 구성 18
[표 26] Auto Scaling 구축 구성 19
[표 27] IAM 구축 구성 20
[표 28] key 생성 구성 21
[표 29] Security Group 구축 구성 21
[표 30] wdb 구축 구성 22
[표 31] WAF 구축 구성 22

Copyright ⓒ 2025 Team_JS. All rights reserved.

그림

[그림 1] 프로젝트 추진 계획 5
[그림 2] 조직별 역할 분담 6
[그림 3] 아키텍쳐 설계도 11

Copyright ⓒ 2025 Team_JS. All rights reserved.

1. 서론

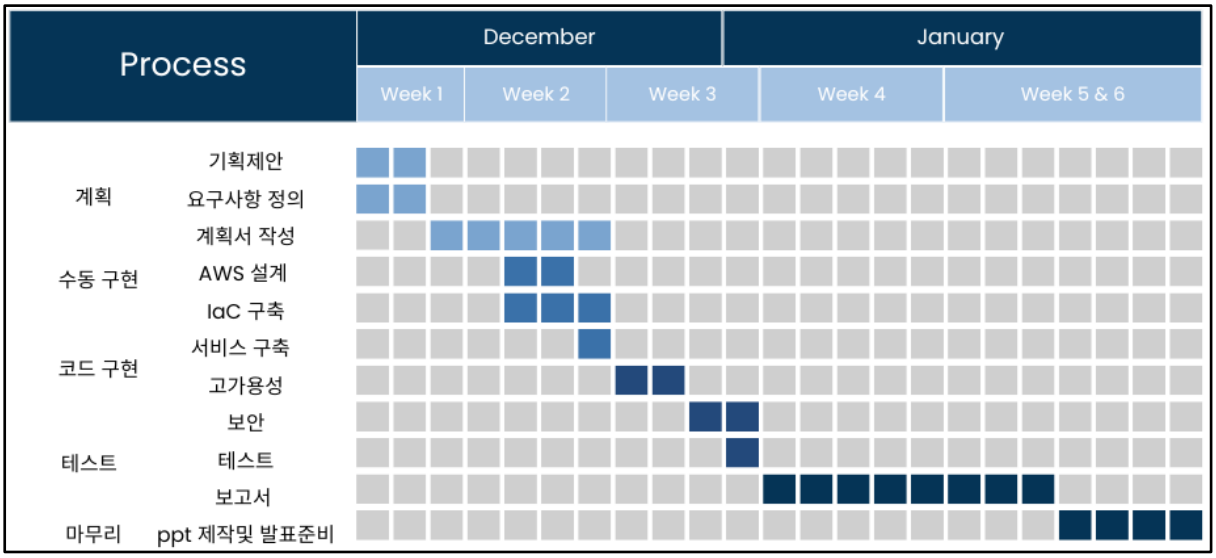
1.1. 프로젝트 개요 및 목적

항목	내용
프로젝트명	Terraform을 활용한 AWS 기반 WordPress 구축
프로젝트 기간	2025.12.17 ~ 2026.01.02
프로젝트 배경	웹 서비스 환경의 복잡성 증가로 수동 인프라 설정에 따른 운영 오류 및 보안 위협이 지속적으로 증가함. 또한 SQL Injection, XSS 등 애플리케이션 계층을 대상으로 한 웹 공격이 고도화됨에 따라 자동화된 인프라 관리와 체계적인 보안 환경 구축의 필요성이 대두됨.
프로젝트 목적	Terraform 기반 IaC를 활용하여 인프라 구성의 일관성과 재현성을 확보함. 로드 밸런서와 Auto Scaling을 통해 안정적인 트래픽 처리 및 고가용성 환경을 구축함. 또한 AWS WAF를 적용하여 웹 애플리케이션 보안 인프라를 강화함.
프로젝트 내용	AWS 서울 리전을 기반으로 VPC, 퍼블릭/프라이빗 서브넷, ALB, Auto Scaling Group, RDS, WAF 등을 포함한 WordPress 웹 서비스 인프라를 코드 기반으로 구축함.
기대 효과	인프라 운영 오류를 최소화함. 트래픽 변화에 유연하게 대응 가능한 고가용성 환경을 확보함. 애플리케이션 계층 보안 강화를 통해 표준화된 클라우드 인프라 운영 체계를 확립함.

[표 1] 프로젝트 개요 및 목적

Copyright ⓒ 2025 Team_JS. All rights reserved.

1.2. 프로젝트 추진 계획



[그림 1] 프로젝트 추진 계획

1.3. 단계별 산출물

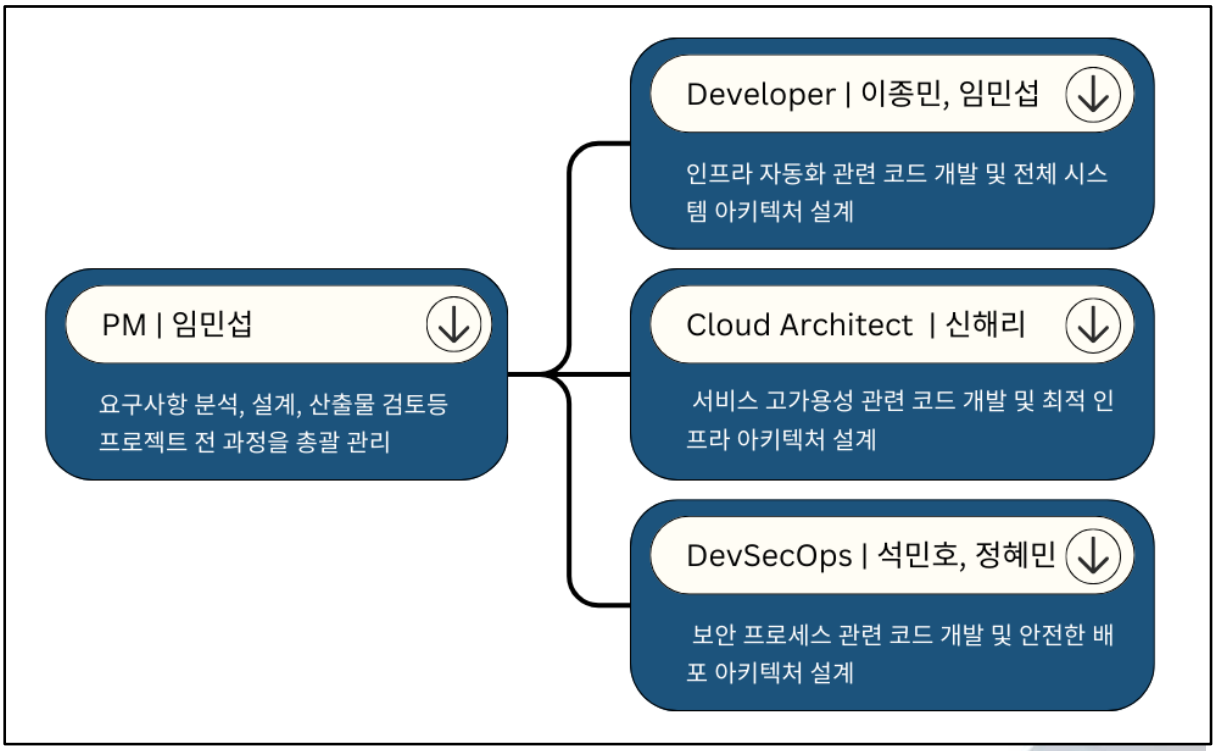
단계	산출물	주요 내용
계획	프로젝트 계획서	- 프로젝트 개요 및 목적 정의 - 수행 일정 및 조직별 역할 분담 수립 - 요구사항 제안 요청서(RFP) 분석 및 정의
구현	Terraform 코드	- 테라폼으로 구현한 인프라 코드화 전체
보고서	최종 완료 보고서	- 아키텍처 설계서 - 단계별 구축 결과 및 인프라 구성 현황 요약 - 인프라 최적화 및 테스트 결과 - 운영 지침 및 향후 개선 방안 제안
발표	PPT	- 발표 자료

[표 2] 단계별 산출물

Copyright ⓒ 2025 Team_JS. All rights reserved.

2. 구축 일정 및 단계

2.1. 프로젝트 수행 조직도



[그림 2] 조직별 역할 분담

Copyright ⓒ 2025 Team_JS. All rights reserved.

3. 기술 및 도구

3.1. Terraform

서비스명	설명
Terraform	HCL 기반 IaC로 AWS 인프라를 코드화하여 일관된 배포 환경을 구성함.

[표 3] 인프라 관리 및 자동화 도구

3.2. AWS 서비스

3.2.1. 인프라 기초 구성

서비스명	설명
VPC	AWS 리전 내에서 논리적으로 격리된 가상 네트워크 환경을 구성함.
Public Subnet	외부 인터넷과 통신이 가능한 서브넷으로 구성함.
Private Subnet	외부 인터넷과 직접 통신하지 않는 내부 전용 서브넷으로 구성함.
Internet Gateway	VPC와 인터넷 간의 통신을 가능하게함
Route Table	서브넷의 트래픽 전달 경로를 정의함.
Route 53	도메인과 IP/AWS 리소스를 매핑하여 DNS 기능을 제공함.
RDS (MariaDB)	관계형 데이터베이스를 관리형으로 제공함.
RDS Snapshot	데이터베이스 상태를 특정 시점 기준으로 백업함.
EC2	가상 머신 형태의 컴퓨팅 자원을 제공함.
AMI	EC2 인스턴스 생성을 위한 운영체제 및 소프트웨어가 포함된 이미지
Launch Template	EC2 인스턴스 생성 설정을 미리 정의함.

[표 4] AWS 인프라 기본 구성 요소

3.2.2. 고가용성 확보

서비스명	설명
Target Group	로드밸런서가 트래픽을 전달할 대상 리소스를 관리함.
Load Balancer	애플리케이션 계층(L7)에서 트래픽을 분산함
Auto Scaling Group	수요 변화에 따라 인스턴스 수를 자동으로 조절함.

[표 5] 고가용성 및 트래픽 처리 구성 요소

Copyright ⓒ 2025 Team_JS. All rights reserved.

3.2.3. 보안 강화

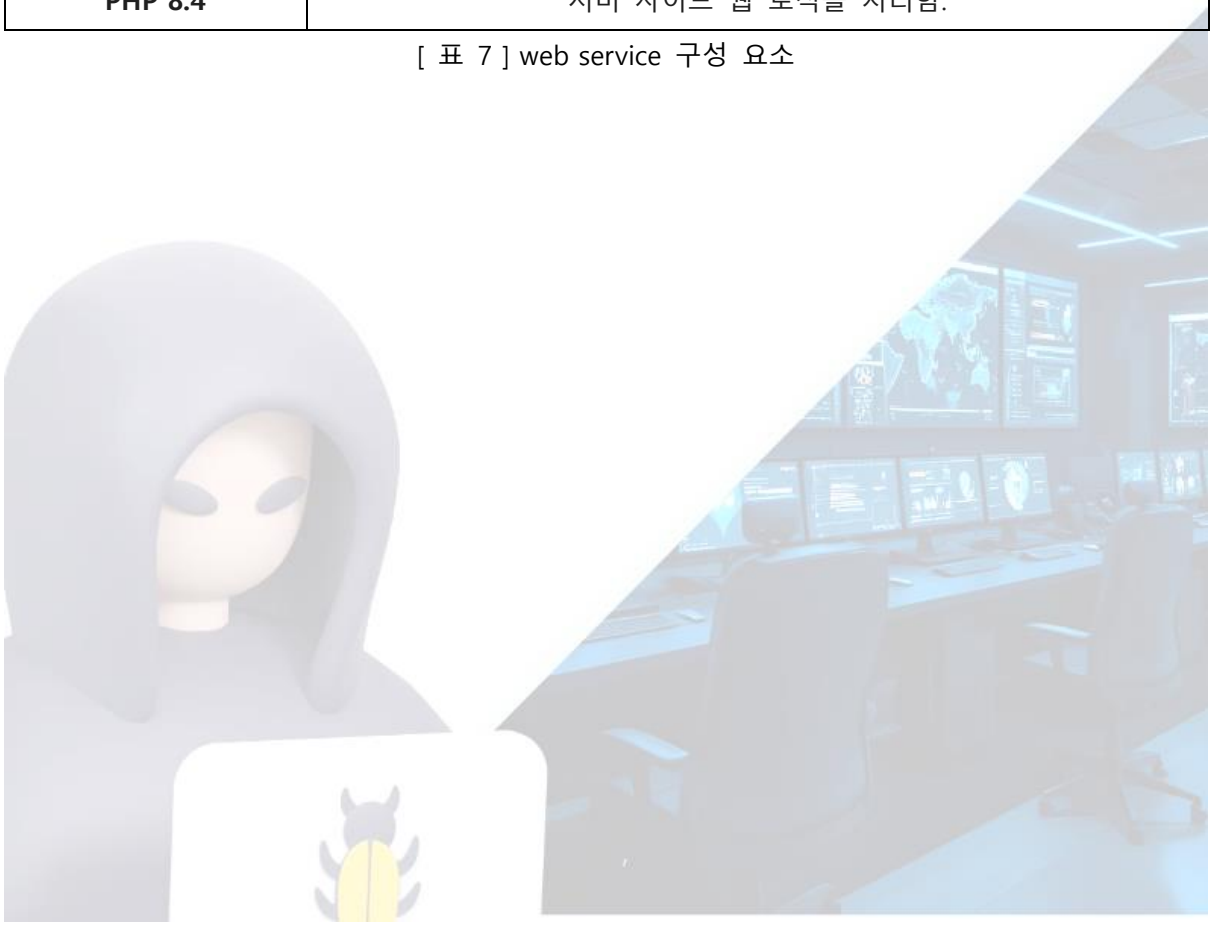
서비스명	설명
WAF	웹 애플리케이션 계층 공격을 필터링함.
Web ACL	WAF에서 적용되는 보안 규칙을 관리함.
Security Group	인스턴스 단위의 네트워크 접근을 제어함.
IAM	AWS 리소스에 대한 사용자 및 권한을 관리함.
Key pair	공개키·개인키 기반의 EC2 인증 수단을 제공함.

[표 6] 보안 처리 구성 요소

3.3. WEB Service

서비스명	설명
WordPress	오픈소스 기반 콘텐츠 관리 시스템(CMS)을 제공함.
Apache (HTTPD)	HTTP 요청을 처리하는 웹 서버 역할을 수행함.
PHP 8.4	서버 사이드 웹 로직을 처리함.

[표 7] web service 구성 요소



Copyright ⓒ 2025 Team_JS. All rights reserved.

4. 요구사항 분석

4.1. 요구사항 제안 요청서 (RFP)와 기술적 대응 방안

4.1.1. 인프라 관련 요구사항

구분	ID	제안 요청 내용	기술적 대응 방안
네트워크	F-01	전사 보안 가이드라인에 따른 가용 영역별 네트워크 분리 및 자원 격리	**VPC & Subnet**: 독립된 VPC를 구성하고 퍼블릭·프라이빗 서브넷을 분리하여 네트워크 격리 환경을 구성함.
통신	F-02	퍼블릭 서브넷 중심의 인터넷 관문 구축 및 비인가 외부 접근 원천 차단	**IGW & Route Table**: 인터넷 게이트웨이를 퍼블릭 서브넷에만 연결하고 라우팅을 제한하여 내부 자원의 외부 직접 접근을 차단함.
도메인	F-03	사용자 편의성 및 시스템 유연성 확보를 위한 가독성 높은 도메인 체계 수립	**Route 53 DNS**: 도메인 기반 DNS 체계를 구성하여 서비스 접근성과 인프라 확장성을 확보함.
데이터베이스	F-04	데이터베이스의 물리적 은닉 및 장애 발생 시 즉각적인 복구 체계 마련	**RDS & Snapshot**: 데이터베이스를 프라이빗 서브넷에 배치하고 스냅샷을 활용한 복구 체계를 구성함.
서비스 처리	F-05	웹 서비스 운영을 위한 컴퓨팅 자원 구성 및 서버 배치 체계 마련	**EC2:** 웹 서비스 실행을 위한 인스턴스를 표준 설정으로 구성하고, 역할/환경에 따라 반복 생성 가능한 서버 배치 구조를 구성함.

[표 8] 인프라 기초 요구사항

Copyright ⓒ 2025 Team_JS. All rights reserved.

4.1.2. 고가용성 관련 요구사항

구분	ID	제안 요청 내용	기술적 대응 방안
트래픽 분산	H-01	대규모 동시 접속자 수용을 위한 부하 분산 처리 및 단일 장애점 제거	**Load Balancer**: 애플리케이션 계층에서 트래픽을 분산하여 단일 장애점을 제거함.
상태 감시	H-02	실시간 리소스 상태 점검을 통한 지능형 장애 인지 및 서비스 연속성 확보	**Health Check**: 주기적인 상태 점검을 통해 비정상 인스턴스를 자동으로 제외함.
자동 확장	H-03	가변적인 트래픽 수요에 대응하는 탄력적 컴퓨팅 자원 운영	**Auto Scaling Group**: 트래픽 변화에 따라 인스턴스 수를 자동으로 확장·축소함.
구성 자동화	H-04	서버 확장 시 환경 일관성 유지를 위한 표준화된 프로비저닝 체계 구축	**Launch Template & AMI**: 표준화된 인스턴스 이미지를 활용하여 동일한 서버 환경을 자동으로 구성함.

[표 9] 고가용성 관련 요구사항

4.1.3. 보안 및 운영 관련 요구사항

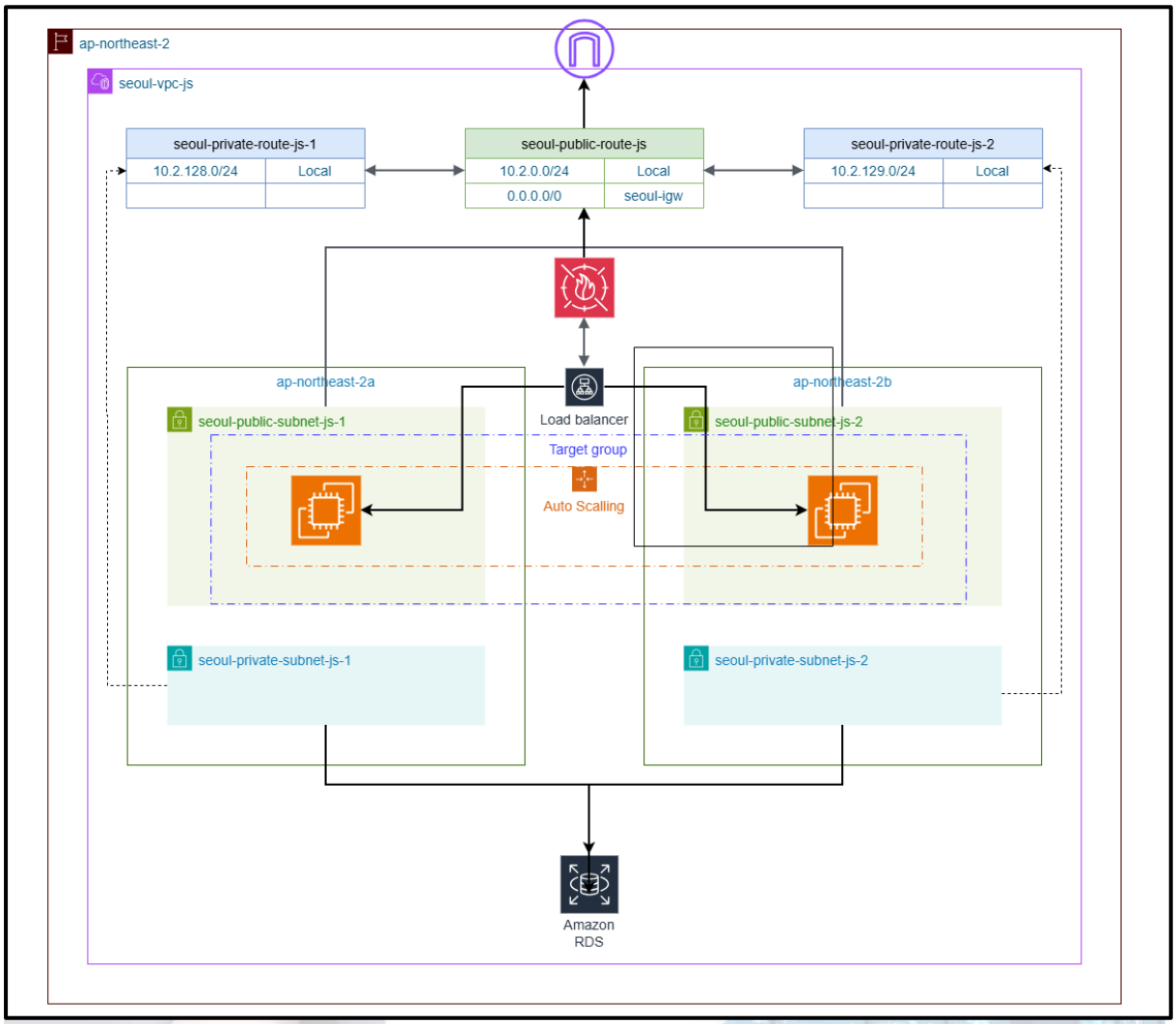
구분	ID	제안 요청 내용	기술적 대응 방안
웹 보안	S-01	외부 웹 공격에 대한 실시간 탐지 및 선제적 차단 체계 구축	**AWS WAF v2**: 웹 공격 패턴을 분석하여 SQL Injection 및 XSS 공격을 차단함.
접근 제어	S-02	시스템 접근 권한의 엄격한 통제 및 인스턴스 단위 방어 강화	**Security Group & Key Pair**: 최소 권한 원칙에 따라 네트워크 접근과 인증을 통제함.
기밀 관리	S-03	민감 정보 유출 방지 및 안전한 자격 증명 관리	**Secrets Manager**: 데이터베이스 접속 정보를 암호화하여 안전하게 관리함.
운영 협업	S-04	다중 작업 환경에서 인프라 상태 일관성 유지	**Remote Backend**: 상태 파일을 공유하고 동시 수정 충돌을 방지함.
감시	S-05	장애 발생 시 신속한 인지 및 관리자 즉각 통보 체계 마련	**CloudWatch**: 리소스 상태를 모니터링하고 임계치 초과 시 알림을 전송함.

[표 10] 보안 및 운영 관련 요구사항

Copyright ⓒ 2025 Team_JS. All rights reserved.

5. 구성

5.1. 전체 아키텍처 설계도



[그림 3] 아키텍처 설계도

AWS 서울 리전(ap-northeast-2)을 기반으로 구성한다.	
퍼블릭 서브넷 구성	**프라이빗 서브넷 구성**
Load Balancer, 타겟 그룹, Auto Scaling Group	데이터베이스 Amazon RDS 배치
LB는 WAF와 연동되어 외부 트래픽을 검사	외부 접근이 차단된 영역이다.
Internet Gateway(IGW)로 향하는 기본 경로 (0.0.0.0/0)를 설정한다.	Internet Gateway 포함 안됨

[표 11] 아키텍처 설계도 설명

Copyright ⓒ 2025 Team_JS. All rights reserved.

5.2. AWS 인프라 구성 설계서

5.2.1. 인프라 구성

▶ Iac 적용

항목	내용
구성 대상	Terraform
Terraform 버전	1.0 이상 2.0 미만
Provider 정보	hashicorp/aws (6.0이상 사용)
사용처	VPC, 서브넷, 보안그룹, EC2, RDS 등 인프라 리소드 코드 정의 및 자동화 배포
구성 방식	모듈화 설계(modules/vpc, modules/keypair, modules/security_group 등)
	변수화(data.yml)관리
배포방식	terraform init -> terraform plan -> terraform apply
설명	Terraform을 통해 인프라 리소스를 코드 기반으로 정의하고 관리하며 변수는 data.yml로 관리하여 yamldecode() 함수를 통해 코드에 전달하는 방식으로 가독성과 관리 편의성을 높이도록 구성

[표 12] IaC 기반 인프라 구축 구성

▶ 리전 구성

항목	내용
구성 대상	Region
리전 코드	ap-northeast-2 (서울)
설명	서울 리전에 리소스를 배포

[표 13] 리전 구축 구성

▶ VPC 구성

항목	내용
구성 대상	VPC
리소스 명	seoul-vpc-js
CIDR 블록	10.2.0.0/16
설명	서울 리전 내 독립적인 VPC 환경 구성

[표 14] VPC 구축 구성

Copyright ⓒ 2025 Team_JS. All rights reserved.

▶ Subnet 구성

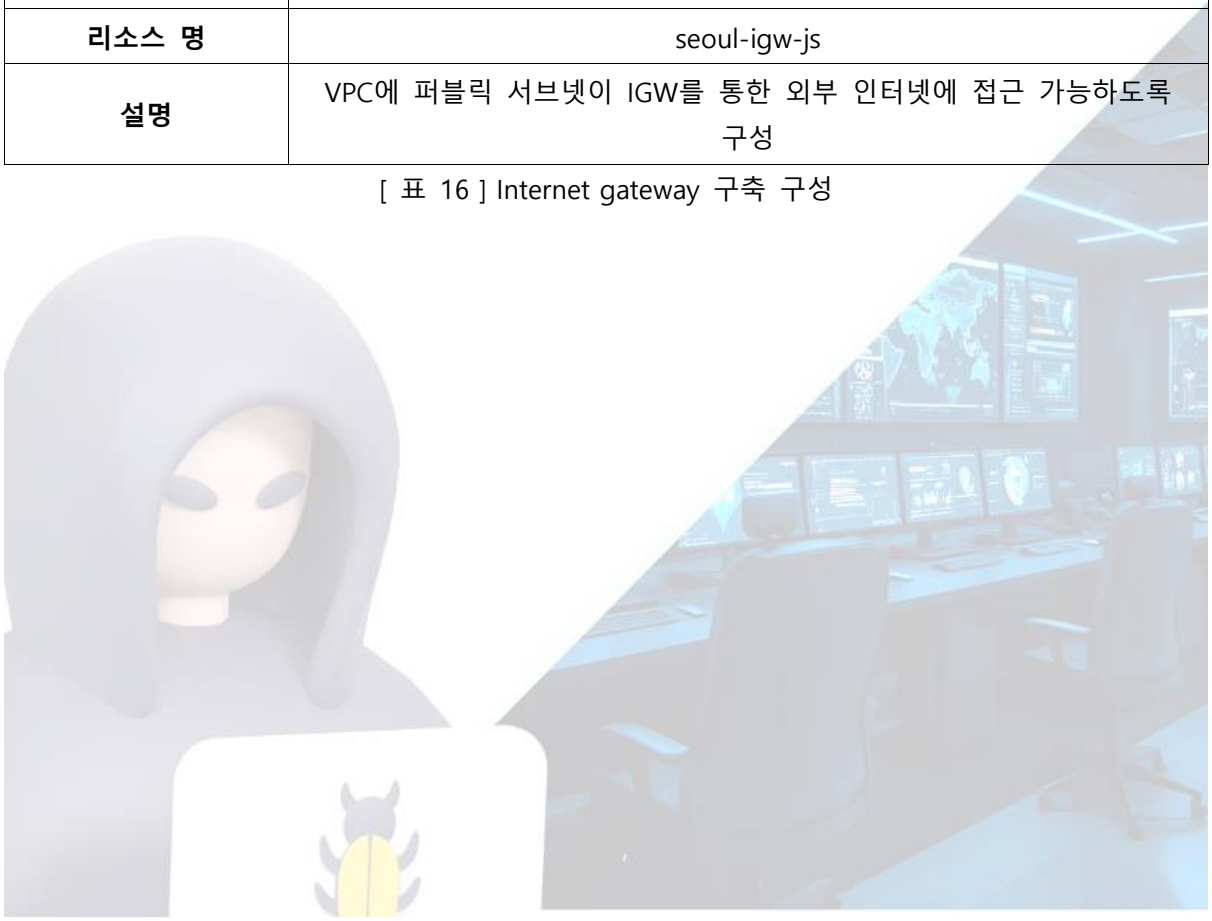
항목	내용	
구성 대상	Subnet	
퍼블릭 서브넷 개수	2개	
퍼블릭 서브넷 명	seoul-public-subnet-js-1	seoul-public-subnet-js-2
CIDR 블록	10.2.0.0/24	10.2.1.0/24
프라이빗 서브넷 개수	2개	
프라이빗 서브넷 명	seoul-private-subnet-js-1	seoul-private-subnet-js-2
CIDR 블록	10.2.128.0/24	10.2.129.0/24
설명	퍼블릭 및 프라이빗 서브넷을 구성하여 외부/내부 접근 서비스가 분리되게 구성	

[표 15] 서브넷 구축 구성

▶ Internet gateway 연결

항목	내용
구성 대상	Internet gateway
리소스 명	seoul-igw-js
설명	VPC에 퍼블릭 서브넷이 IGW를 통한 외부 인터넷에 접근 가능하도록 구성

[표 16] Internet gateway 구축 구성



Copyright ⓒ 2025 Team_JS. All rights reserved.

▶ 라우팅 구성

항목	내용	
구성 대상	Route Table	
퍼블릭 라우팅 테이블 명	seoul-route-public	
연결된 서브넷	seoul-public-subnet-js-1	seoul-public-subnet-js-2
라우팅 경로	10.2.0.0/24 -> local	10.2.1.0/24 -> local
연결된 리소스	seoul-igw (Internet Gateway)	
라우팅 경로	0.0.0.0/0 -> seoul-igw-js	
설명	퍼블릭 서브넷은 1개의 퍼블릭 라우팅 테이블을 공유하고 IGW에 연결하여 외부와 통신할 수 있도록 구성	
프라이빗 라우팅 테이블 명	seoul-route-private-js-1	seoul-route-private-js-2
연결된 서브넷	seoul-private-subnet-js-1	seoul-private-subnet-js-2
라우팅 경로	10.2.0.0/16 -> local	10.2.0.0/16 -> local
설명	프라이빗 서브넷은 각 서브넷별 라우팅 테이블을 분리하여 구성하고, 동일 NAT Gateway를 통해 외부와 통신할 수 있도록 구성	

[표 17] Route Table 구축 구성

▶ 도메인 구성

항목	내용	
구성 대상	Route 53	
서비스 구분	Wordpress	RDS
Zone 타입	퍼블릭	프라이빗
도메인 명	www.js.it-edu.org	database.js.it-edu.org
설명	퍼블릭 웹 서비스 도메인 관리용 호스팅존을 구성	

[표 18] 도메인 구축 구성

Copyright ⓒ 2025 Team_JS. All rights reserved.

▶ RDS 구성

항목	내용
구성 대상	RDS Databases
DB 엔진 및 버전	MariaDB 11.4.5
DB 인스턴스 명	mariadb-ec2
인스턴스 타입	db.t4g.micro
username/password	root/Passw0rd
스토리지 타입	gp3
스토리지 크기	20GB
DB 서브넷 그룹 명	mysql-subnet-group
서브넷 그룹 구성	프라이빗 서브넷 2개 지정 (seoul-route-private-1, seoul-route-private-2)
보안 그룹	seoul-sg-js-rds (인바운드 규칙 3306 개방)
퍼블릭 접근 여부	비활성화(False)
스냅샷 복원	wordpress-db
최종 스냅샷 생성 여부	비활성화(False)
삭제 방지	비활성화(False)
설명	초기 데이터 및 구성 상태는 지정된 스냅샷을 기준으로 복원 ※ 추가 데이터나 구성 변경 사항이 있을 경우 반드시 새 스냅샷을 생성하고 최신 스냅샷으로 업데이트 해야한다.

[표 19] RDS 구축 구성

Copyright ⓒ 2025 Team_JS. All rights reserved.

▶ RDS 연동

항목	내용
구성 대상	Amazon RDS
RDS 명	seoul-db-js
도메인	database.js.it-edu.org
RDS 접근 방식	웹 서버에서 mariadb-client 를 사용해 접근
연결 경로	웹 서버(EC2) → VPC 내부 라우팅 → 프라이빗 서브넷에 위치한 Amazon RDS
설명	Amazon RDS는 서울 리전의 프라이빗 서브넷에 배포하여 외부 인터넷으로부터 직접 접근을 차단하였다. 데이터베이스 접근은 동일 VPC 내 웹 서버(EC2)에서만 허용되도록 보안 그룹을 통해 제어하였다.

[표 20] RDS 연동 구성

▶ EC2 인스턴스 구성

항목	내용
구성 대상	EC2 Instance
EC2 용도	WEB 서버
EC2 인스턴스 명	seoul-ec2-js
AMI 명	project-mega
OS 이미지	Amazon Linux 2023 AMI
인스턴스 타입	t3.micro
SSH 키페어 이름	seoul-rsa-key-js
VPC 명	seoul-vpc-js
서브넷 명	seoul-public-subnet-js-1
퍼블릭 IP 자동 할당	활성화(True)
보안 그룹 명	seoul-sg-js-web

[표 21] EC2 인스턴스 구축 구성

Copyright ⓒ 2025 Team_JS. All rights reserved.

▶ AMI 이미지 관리

항목	내용
구성 대상	AMI
AMI 용도	WEB 서버용
AMI 명	project-mega
OS 이미지	Amazon Linux 2023 AMI
인스턴스 타입	t3.micro
SSH 키페어 이름	seoul-rsa-key-js
VPC 명	seoul-vpc-js
서브넷 명	seoul-public-subnet-js-1
퍼블릭 IP 자동 할당	활성화(True)
보안 그룹 명	web

[표 22] AMI 구축 구성

5.2.2. 고가용성 확보

▶ 가용 영역(AZ) 분산 구성

항목	내용
구성 대상	Availability Zone (AZ)
사용 가용영역(AZ)	ap-northeast-2
퍼블릭 서브넷 분배	seoul-public-subnet-js-1 seoul-public-subnet-js-2
프라이빗 서브넷 분배	seoul-private-subnet-js-1 seoul-private-subnet-js-2
설명	서울 리전(ap-northeast-2)의 가용영역 중 2개(AZ)를 선택하여 서브넷과 리소스를 균등하게 분산 배치함, 이를 통해 리소스의 고가용성 확보

[표 23] 가용 영역 분산 구성

Copyright ⓒ 2025 Team_JS. All rights reserved.

▶ Target group 구성

항목	내용
구성 대상	Target Group
대상 유형	Instance
적용 서비스	WEB 서버
프로토콜 / 포트	HTTP / 80
헬스 체크 경로	/index.php
적용 VPC	seoul-vpc-js
연동 Load Balancer	seoul-loadbalancer-js
설명	로드밸런서가 트래픽을 전달할 백엔드 EC2 인스턴스 그룹을 정의하고, 헬스 체크를 통해 정상 상태의 인스턴스에만 트래픽이 전달되도록 구성함.

[표 24] TG 구축 구성

▶ Launch Template 구성

항목	내용
구성 대상	Launch Template
탬플릿 명	seoul-template-js
기준 AMI	project-mega
인스턴스 타입	t3.micro
SSH 키페어 이름	seoul-rsa-key-js
보안그룹	seoul-sg-web
네트워크 인터페이스	퍼블릭 IP 자동 할당 : 활성화(True) 인스턴스 종료시 삭제 : 활성화(True)

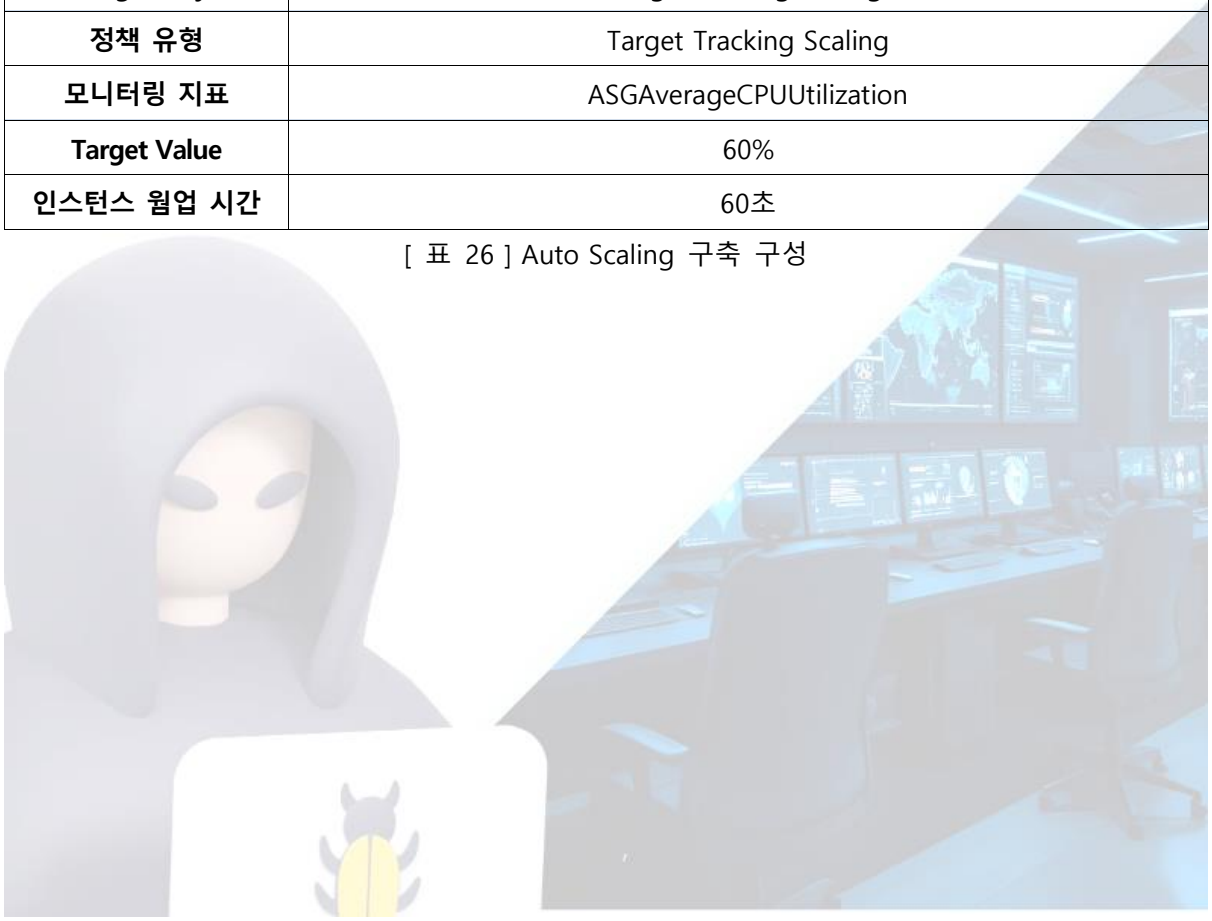
[표 25] Template 구축 구성

Copyright ⓒ 2025 Team_JS. All rights reserved.

▶ Auto Scaling

항목	내용
구성 대상	autoscaling_group
리소스 명	wordpress
Launch Template	template-ec2-wordpress-dynamic
최소 인스턴스 수	2
최대 인스턴스 수	4
Desired Capacity	2
서브넷	seoul-public-subnet-js-1 seoul-public-subnet-js-2
헬스체크 유형	ELB
헬스체크 유예 시간	900초
유지관리 정책	최소 정상 비율 : 90% 최대 정상 비율: 100%
Scaling Policy 명	TargetTrackingScailing
정책 유형	Target Tracking Scaling
모니터링 지표	ASGAverageCPUUtilization
Target Value	60%
인스턴스 웜업 시간	60초

[표 26] Auto Scaling 구축 구성



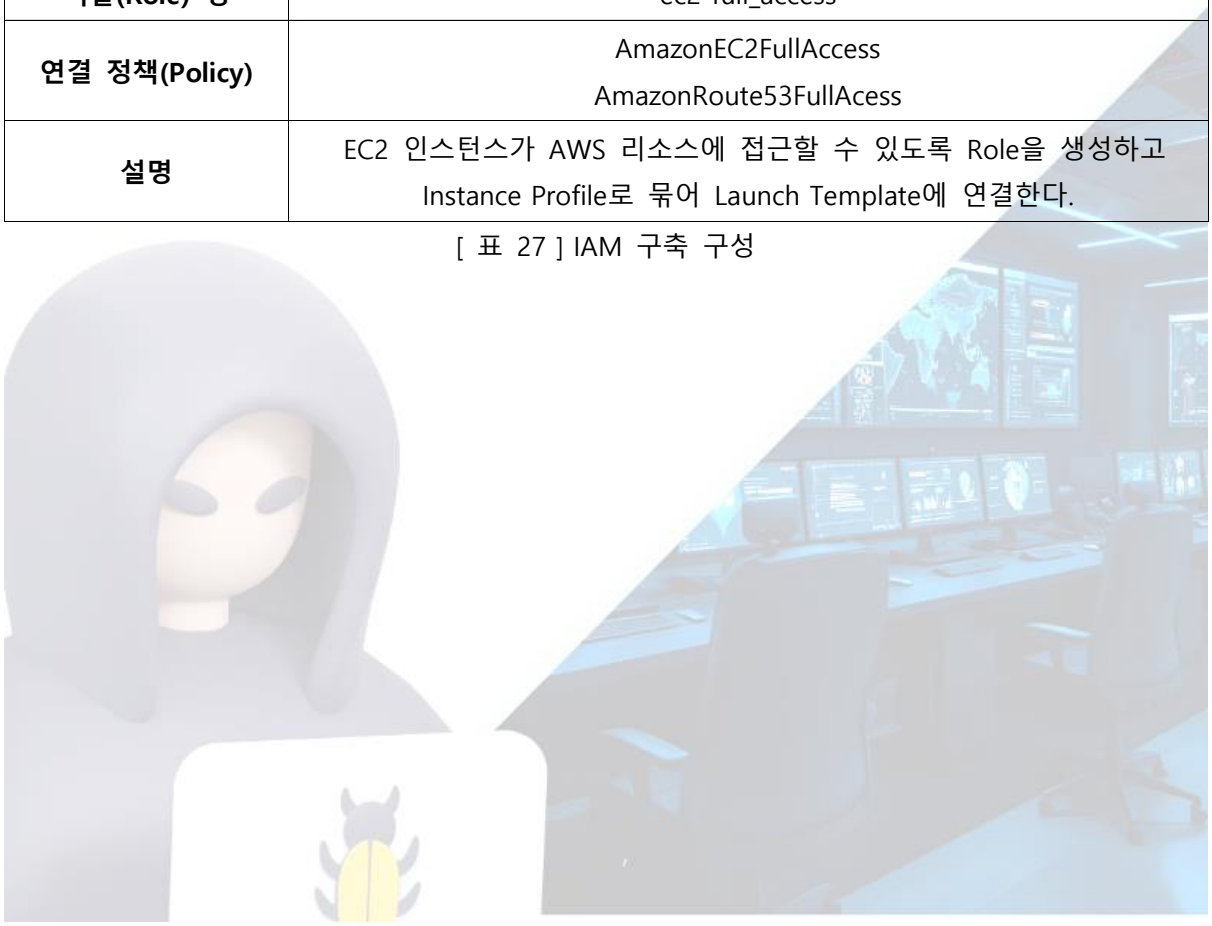
Copyright ⓒ 2025 Team_JS. All rights reserved.

5.2.3. 보안 강화

▶ **IAM 구성**

항목	내용
구성 대상	IAM Users
사용자(User) 명	terraform
적용 정책(Policy)	AmazonEC2FullAccess AmazonLaunchWizardFullAccessV2 AmazonRDSFullAccess AmazonRoute53FullAcess AmazonVPCFullAccess AutoScalingFullAccess EC2FastLaunchFullAccess
설명	IAM 사용자(Terraform)을 생성하여 Terraform 및 관리용 계정으로 사용
리소스 유형	IAM Roles
역할(Role) 명	ec2-full_access
연결 정책(Policy)	AmazonEC2FullAccess AmazonRoute53FullAcess
설명	EC2 인스턴스가 AWS 리소스에 접근할 수 있도록 Role을 생성하고 Instance Profile로 묶어 Launch Template에 연결한다.

[표 27] IAM 구축 구성



Copyright ⓒ 2025 Team_JS. All rights reserved.

▶ key 생성

항목	내용
구성 대상	Key Pairs
사용처	terraform 코드로 배포된 리소스용
퍼블릭 키 페어 명	seoul-rsa-key-js
알고리즘 / 비트수	RSA / 2048
파일/디렉토리 권한	0600 / 0700
설명	Terraform 코드에서 자동으로 생성하고 비밀키는 로컬 모듈 경로에 저장

[표 28] key 생성 구성

▶ 보안 설정

항목	내용				
구성 대상	Security Group				
보안그룹 명	seoul-sg-web				
방향	ingress				egress
서비스 명	https	http	ssh	echo	all_outbound
프로토콜	tcp	tcp	tcp	icmp	-1
포트번호	443	80	22	type 8	0
CIDR 블럭	0.0.0.0/0				0.0.0.0/0
보안그룹 명	seoul-sg-js-rds				
방향	ingress				egress
서비스 명	MYSQL/Aurora				all_outbound
프로토콜	TCP				-1
포트번호	3306				0
CIDR 블럭	0.0.0.0/0				0.0.0.0/0
보안그룹 명	seoul-sg-lb				
방향	ingress				egress
서비스 명	SSH		HTTP		all_outbound
프로토콜	TCP		TCP		-1
포트번호(type)	22		80		0
CIDR 블럭	0.0.0.0/0				0.0.0.0/0

[표 29] Security Group 구축 구성

Copyright ⓒ 2025 Team_JS. All rights reserved.

▶ 웹 서비스 구성

항목	내용
구성 대상	Web Application
용도	Wordpress(게시판)
도메인	www.js.it-edu.org
RDS 연동	RDS 스냅샷 사용
로그 수집 방식	WAF의 ACL를 이용한 로그 수집
AMI 초기설정	- wordpress 설치 - 웹 관련 패키지 설치

[표 30] wdb 구축 구성

▶ WAF

항목	내용
구성 대상	AWS WAF (Web ACL)
적용 대상	Application Load Balancer
보호 서비스	WordPress 웹 서비스
적용 도메인	www.js.it-edu.org
적용 리전	ap-northeast-2 (Seoul)
주요 보호 목적	웹 애플리케이션 계층 공격 탐지 및 차단
적용 규칙	AWS Managed Rule (SQL Injection, XSS 등)
연동 구조	외부 트래픽 → WAF → ALB → EC2
로그 수집 방식	Web ACL 로그를 통해 요청 정보 기록
로그 활용 목적	공격 시도 분석 및 보안 이벤트 추적
설명	WAF를 Load Balancer 앞단에 적용하여 SQL Injection, XSS 등 웹 공격을 사전에 차단하고, 백엔드 서버 및 데이터베이스를 보호하도록 구성함.

[표 31] WAF 구축 구성

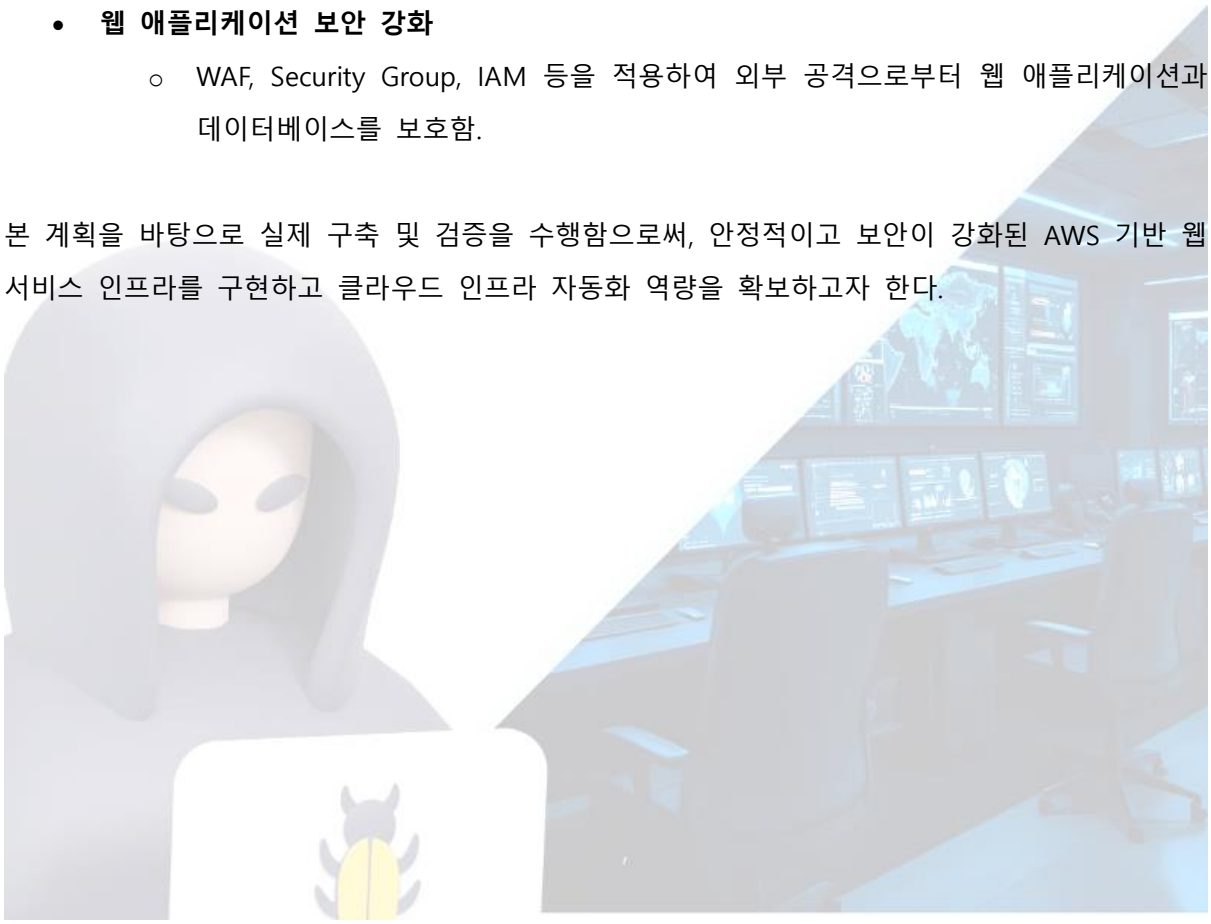
Copyright ⓒ 2025 Team_JS. All rights reserved.

6. 결론 및 목표

본 프로젝트는 Terraform 기반 IaC를 활용하여 AWS 환경에서 WordPress 웹 서비스를 **표준화·자동화된 방식으로 구축**하는 것을 목표로 한다.
본 계획을 통해 다음과 같은 효과를 기대할 수 있다.

- **IaC 기반 인프라 표준화**
  - Terraform을 활용하여 인프라를 코드로 관리함으로써 구성 오류를 최소화하고 재현 가능한 배포 환경을 확보함.
- **보안성과 가용성을 고려한 인프라 설계**
  - 퍼블릭·프라이빗 서브넷 분리 및 가용 영역(AZ) 분산 배치를 통해 전사 보안 가이드라인을 준수하고 안정적인 서비스 환경을 구성함.
- **트래픽 대응 및 확장성 확보**
  - Load Balancer와 Auto Scaling 구조를 통해 트래픽 변화에 유연하게 대응 가능한 웹 서비스 처리 환경을 설계함.
- **웹 애플리케이션 보안 강화**
  - WAF, Security Group, IAM 등을 적용하여 외부 공격으로부터 웹 애플리케이션과 데이터베이스를 보호함.

본 계획을 바탕으로 실제 구축 및 검증을 수행함으로써, 안정적이고 보안이 강화된 AWS 기반 웹 서비스 인프라를 구현하고 클라우드 인프라 자동화 역량을 확보하고자 한다.



Copyright ⓒ 2025 Team_JS. All rights reserved.